Foundations of Deep Learning

Lecture 07

Reinforcement Learning

Aurelien Lucchi

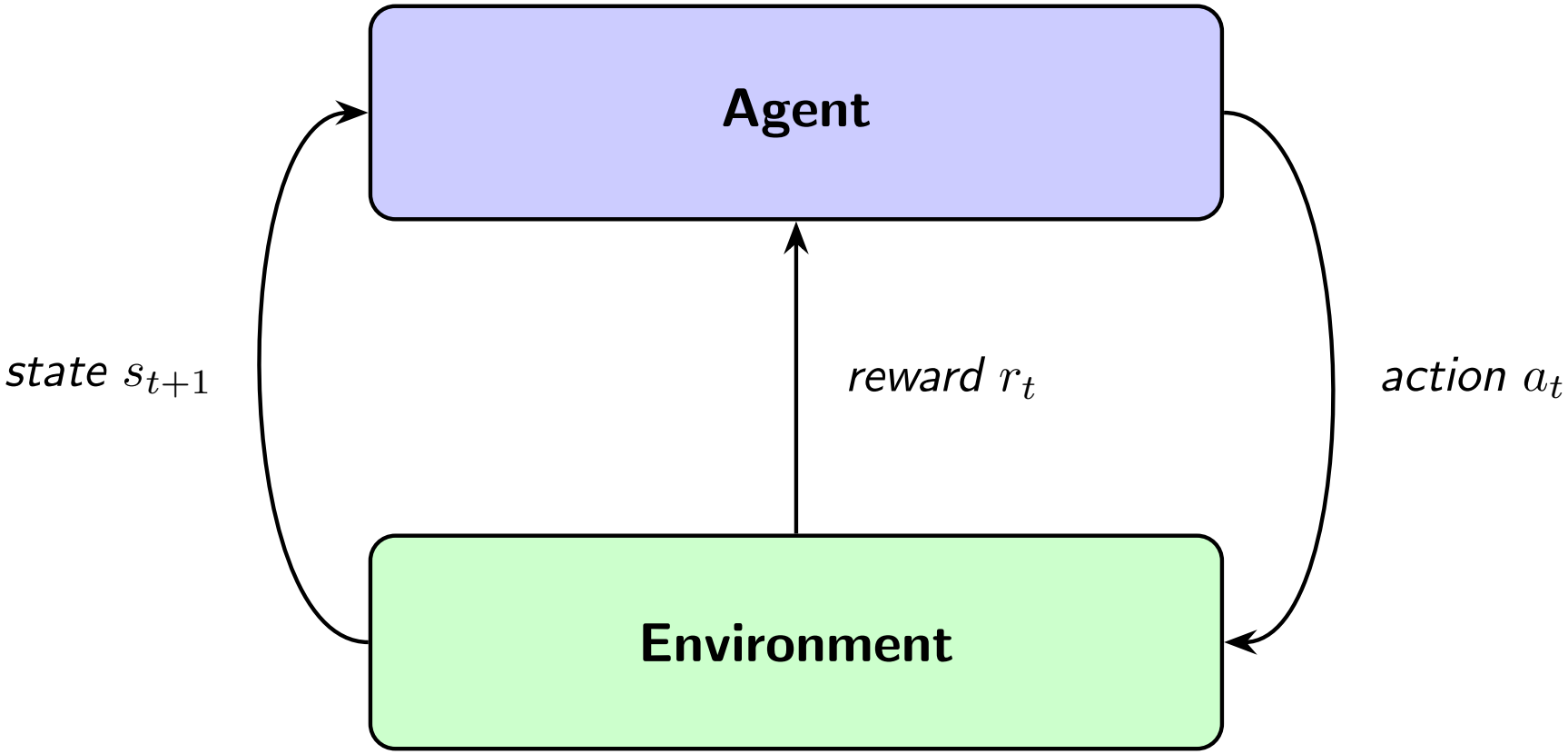
Fall 2026

What is Reinforcement Learning (RL)?

Definition RL is a paradigm in which an agent learns to make a **sequence of decisions** by interacting with an environment and **receiving feedback** in the form of rewards to maximize its long-term cumulative return.

Deep RL = RL + Deep Network

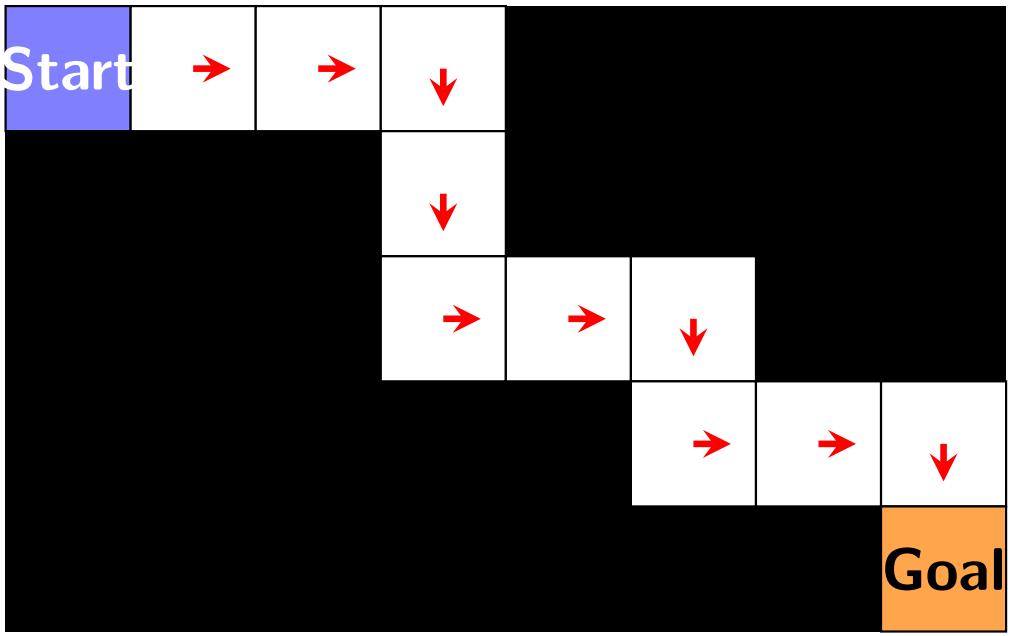
Agent–Environment Interaction



At each time t, the agent observes s_t, takes a_t, receives r_t, and the environment transitions to s_{t+1}.

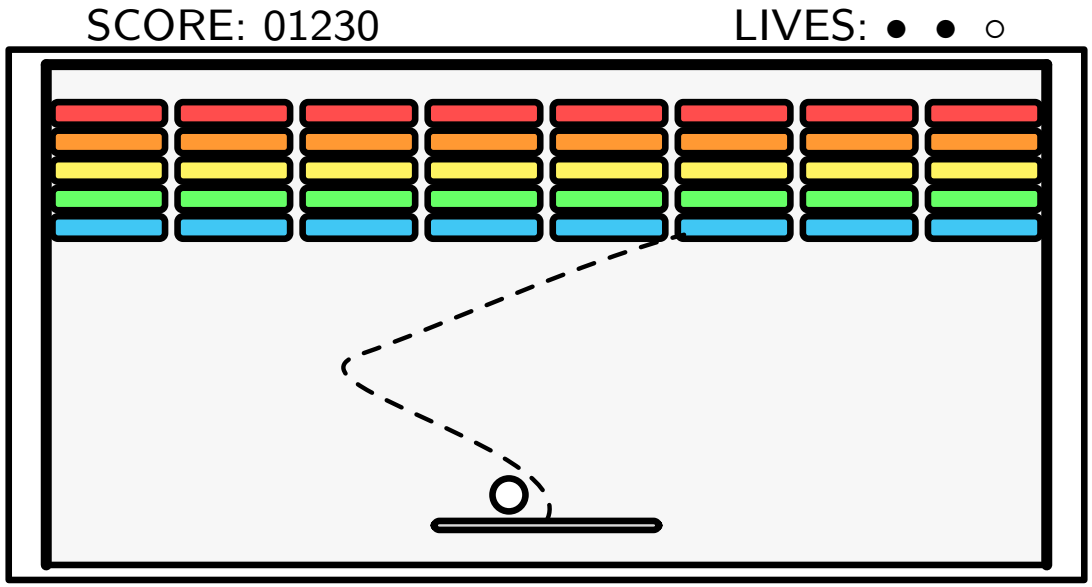
Examples: Maze-Navigating Agent

- **Environment:** Grid of free cells and walls; one start and one goal.
- **State:** Agent's coordinates in the grid.
- **Actions:** Move {up, down, left, right} if not blocked.
- **Reward:** $+10$ at goal; -1 per step; -5 for hitting a wall.



Examples: Atari Breakout

- **Environment:** The game screen with paddle, ball, bricks.
- **State:** Recent frames (image stack) capturing positions/velocities.
- **Actions:** {left, right, no-op} (sometimes `fire`).
- **Reward:** $+1$ per brick; $+50$ for clearing a level; -1 or termination on losing a life.



Supervised vs Unsupervised vs RL

Aspect	**Supervised**	**Unsupervised**	**RL**
Goal	Map inputs to known outputs	Discover structure	Maximize cumulative reward via interaction
Training Data	Labeled (x_i, y_i)	Unlabeled $\{x_i\}$	Interaction tuples (s_t, a_t, r_t, s_{t+1})
Feedback	Direct labels	None (implicit)	Delayed, evaluative (rewards)
Output	Label/value	Clusters / density	Policy $\pi^*(a \mid s)$ or $v^*(s)$

Definition 1 (Discounted cumulative reward)

$$\begin{aligned} G_t &:= r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \cdots \\ &= r_t + \gamma G_{t+1}\,, \end{aligned}$$

with discount factor $0 \leq \gamma < 1$.

Remark 1 (Why discounting?)

Ensures finiteness: $\sum_{k=0}^{\infty} \gamma^k = \frac{1}{1-\gamma}$.

Goal: learn a policy that maximizes expected return.

Markov Decision Process (MDP)

Definition 2

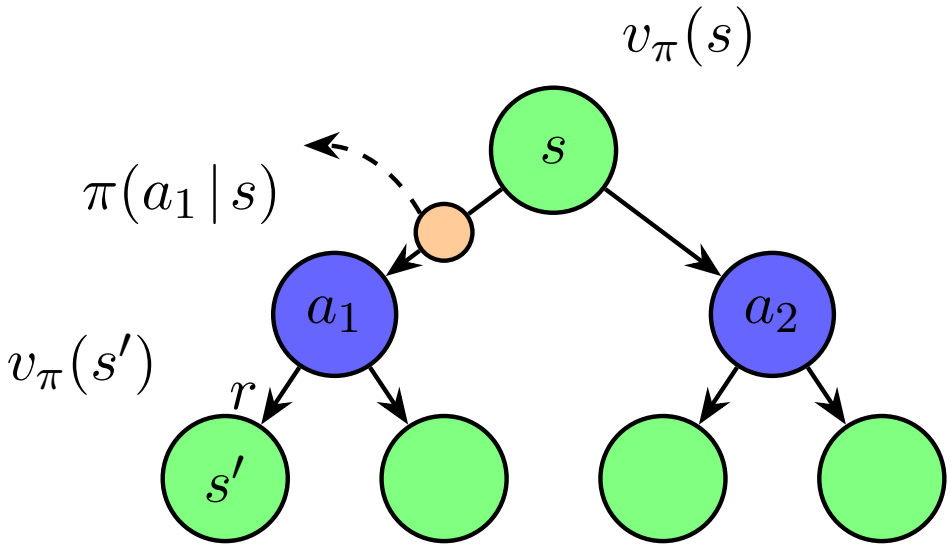
An MDP is a 5-tuple (S, A, P, R, γ):

- S: states, A: actions,
- $p(r, s' \mid s, a)$: transition probabilities,
- $r(s, a)$: immediate (expected) reward,
- $\gamma \in [0, 1)$: discount factor.

Value (state-value) function:

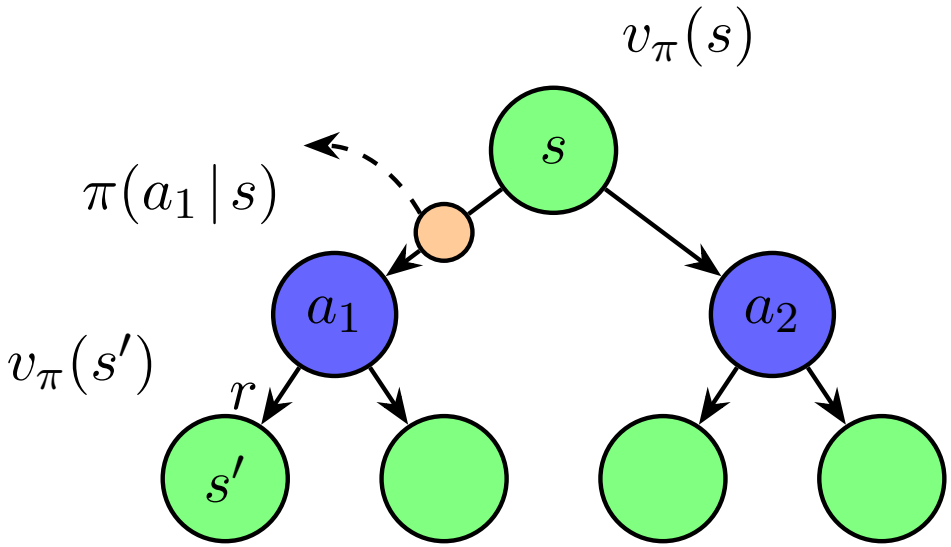
$$v_\pi(s) = \mathbb{E}_\pi[G_t \mid S_t = s] = \mathbb{E}_\pi[r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \cdots \mid S_t = s],$$

where the expectation is taken over the joint distribution of all actions, states and rewards starting from t until the end of an episode.



Bellman expectation equation:

$$
\begin{aligned}
v_\pi(s) &= \mathbb{E}_\pi[r_t + \gamma G_{t+1} \mid S_t = s] \\
&= \sum_a \pi(a \mid s) \sum_{r,s'} p(r, s' \mid s, a)[r + \gamma \mathbb{E}_\pi[G_{t+1} \mid S_{t+1} = s']] \\
&= \sum_a \pi(a \mid s) \sum_{r,s'} p(r, s' \mid s, a)[r + \gamma v_\pi(s')] \\
&= \mathbb{E}_\pi \left[r_t + \gamma v_\pi(s_{t+1})\right].
\end{aligned}
$$



Action-value function:

$$q_\pi(s, a) = \mathbb{E}_\pi[G_t \mid S_t = s, A_t = a]\,.$$

Bellman expectation (for q_π):

$$\begin{aligned} q_\pi(s, a) &= \sum_{r,s'} p(r, s' \mid s, a)\,\big[r + \gamma\, v_\pi(s')\big] \\ &= \mathbb{E}\big[r_t + \gamma\, v_\pi(S_{t+1}) \mid S_t = s, A_t = a\big]. \end{aligned}$$

Relation to v_π:

$$v_\pi(s) = \sum_{a \in \mathcal{A}} \pi(a \mid s)\, q_\pi(s, a).$$

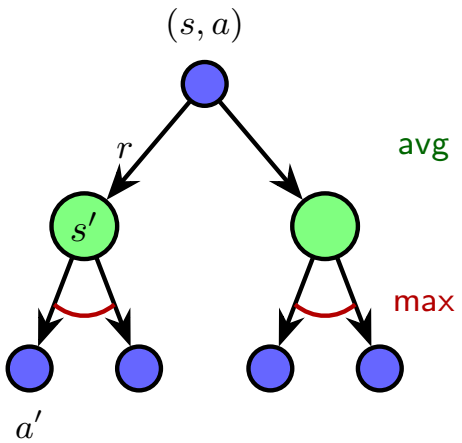
Bellman Optimality Equation

Optimal value functions: the best achievable by any policy,
$v_*(s) = \max_\pi v_\pi(s), \quad q_*(s,a) = \max_\pi q_\pi(s,a).$

$$q_*(s,a) = \mathbb{E}\big[r_t + \gamma\, v_*(s_{t+1}) \mid s_t = s, a_t = a\big] \tag{1}$$

$$v_*(s) = \max_a q_*(s,a) \tag{2}$$

$$\Rightarrow \quad q_*(s,a) = \mathbb{E}\big[r_t + \gamma \max_{a'} q_*(s_{t+1}, a') \mid s_t = s, a_t = a\big] \quad (1) + (2)$$

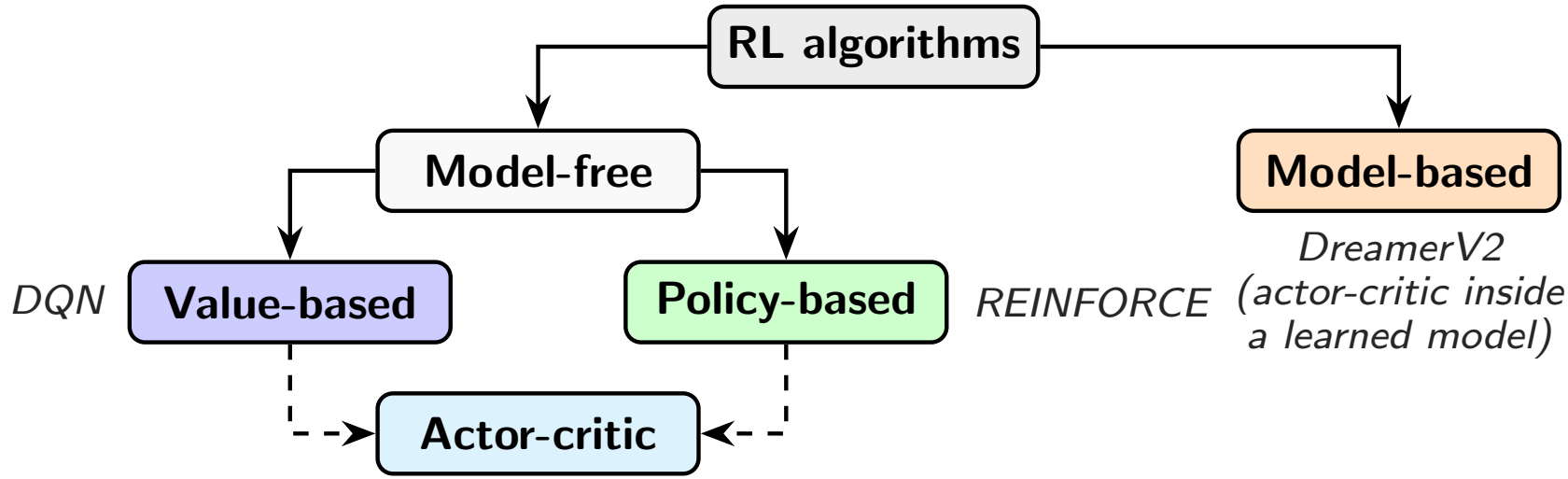


Proof of (2). "$\le$": averaging never beats the best action, $\sum_a \pi(a \mid s)\, q_*(s,a) \le \max_a q_*(s,a)$.
"$\ge$": let π_* be an optimal policy and $a^\star = \arg\max_a q_*(s,a)$. Consider the strategy "take $a^\star$ now, then follow π_*". Its expected return from s is $\mathbb{E}\big[r_t + \gamma\, v_*(s_{t+1}) \mid s_t = s, a_t = a^\star\big] = q_*(s,a^\star)$ by (1). Since $v_*(s)$ is the best achievable from s, it is at least as good as this strategy:

$$v_*(s) \;\ge\; q_*(s,a^\star) = \max_a q_*(s,a).$$

Learning Algorithms: Overview

Algorithms differ in **what is learned** and **how actions are chosen**.



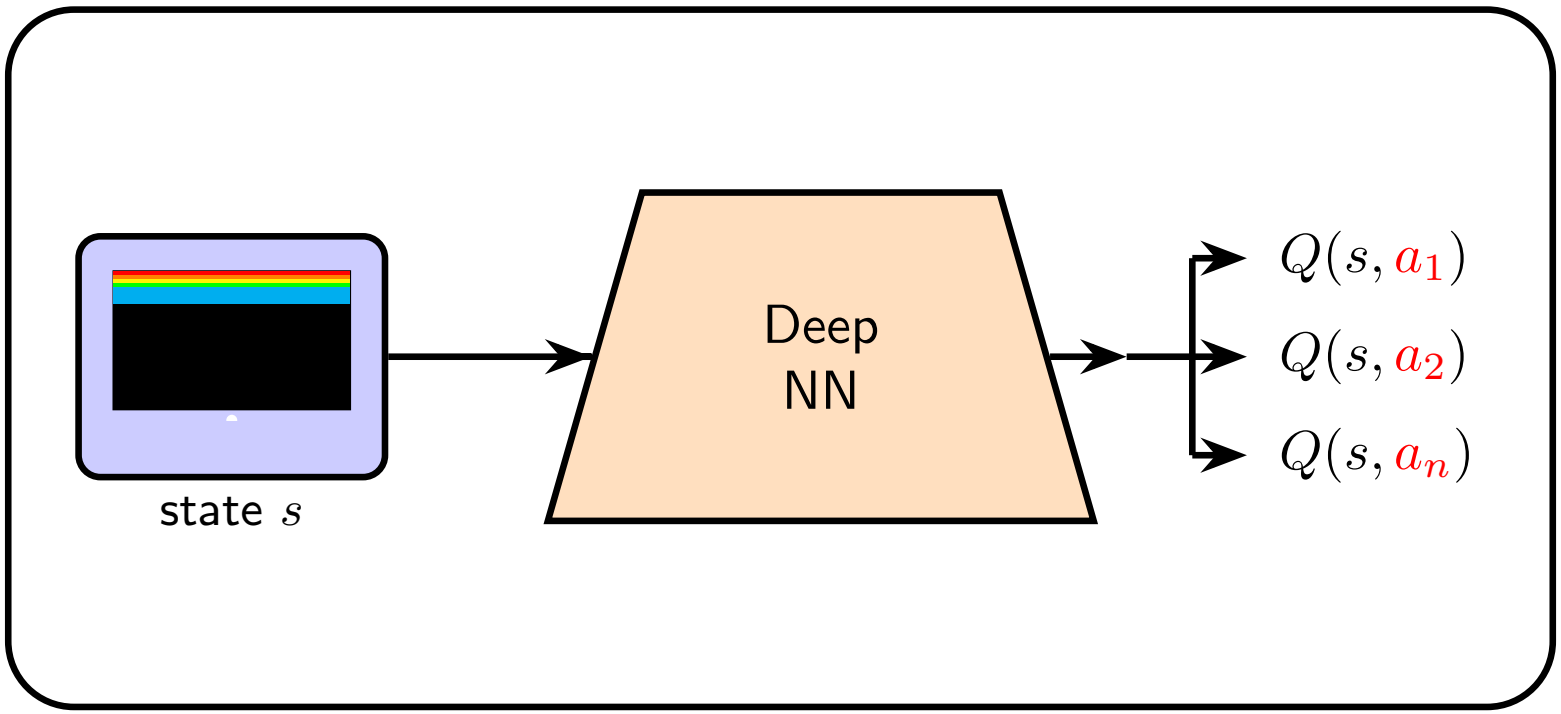
A2C, GAE (learns both a policy π_θ (actor) and a value V_ψ (critic))

	What is learned	How to act
Value-based	q_*, via Bellman optimality: $q_*(s,a) = \mathbb{E}\big[r + \gamma \max_{a'} q_*(s',a')\big]$	greedy: $\pi_*(s) = \arg\max_a q_*(s,a)$
Policy-based	π_θ directly, by gradient ascent on $J(\theta) = \mathbb{E}_{\pi_\theta}[G_t]$	sample $a \sim \pi_\theta(\cdot \mid s)$
Model-based	the environment itself: $\hat{p}_\phi(r, s' \mid s, a) \approx p(r, s' \mid s, a)$	plan, or train π_θ on imagined experience

Classical

- Q-learning: target $r + \gamma \max_{a'} Q(s', a') \Rightarrow Q$ estimates q_*, even while the agent explores (off-policy).
- SARSA: target $r + \gamma Q(s', a')$, $a' =$ action actually taken next $\Rightarrow Q$ estimates q_π of the policy followed (on-policy).

Deep Q-Learning (DQN) = Q-learning with a neural network



On-Policy vs Off-Policy Learning

Question: is the policy we **learn about** the same as the policy that **collects the data**?

	On-policy	**Off-policy**
Same policy?	yes	no
Data comes from	the current policy	any policy (e.g. older)
Reuse old data?	no	yes $\Rightarrow$ replay buffer
Examples	SARSA, REINFORCE	Q-learning, DQN

(1) Network architecture.

- Input: raw pixel observations
- Output: vector of Q-values, one per action.
- Typically a network with a few layers

(2) Experience replay.

- Transitions (s_t, a_t, r_t, s_{t+1}) are stored in a **replay memory/buffer** $\mathcal{D}$
- During training, mini-batches of transitions are sampled uniformly from $\mathcal{D}$ to perform updates: $(s, a, r, s') \sim \mathcal{D}$

(3) Target network. A separate **target network** $Q(s, a; \theta^-)$ is used to compute the target value:

$$y_t = r_t + \gamma \max_{a'} Q(s_{t+1}, a'; \theta^-). \tag{1}$$

The main network parameters θ are optimized to minimize the squared error:

$$L(\theta) = \mathbb{E}_{(s,a,r,s')\sim\mathcal{D}}\left[\left(y_t - Q(s,a;\theta)\right)^2\right]. \tag{2}$$

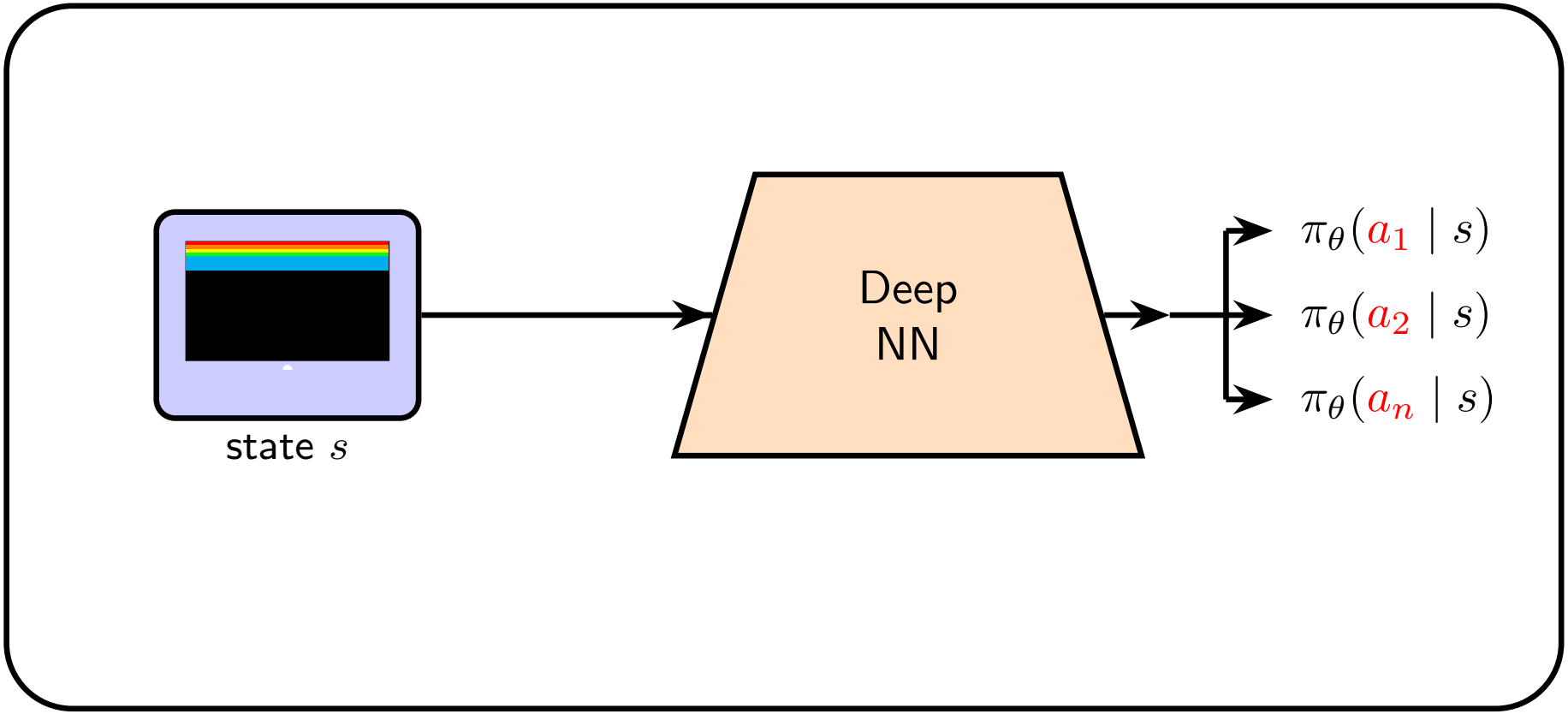
Every C steps, the target network is updated by copying the weights: $\theta^- \leftarrow \theta$.

(4) Optimization. SGD using the gradient:

$$\nabla_\theta L(\theta) = \mathbb{E}[(Q(s,a;\theta) - y_t)\,\nabla_\theta Q(s,a;\theta)]\,.$$

(5) Exploration. DQN uses an ϵ-greedy exploration strategy: with probability ϵ, a random action is chosen, and with probability $1-\epsilon$, the greedy action $\arg\max_a Q(s,a;\theta)$ is taken. The exploration rate ϵ decays over time.

Policy Learning

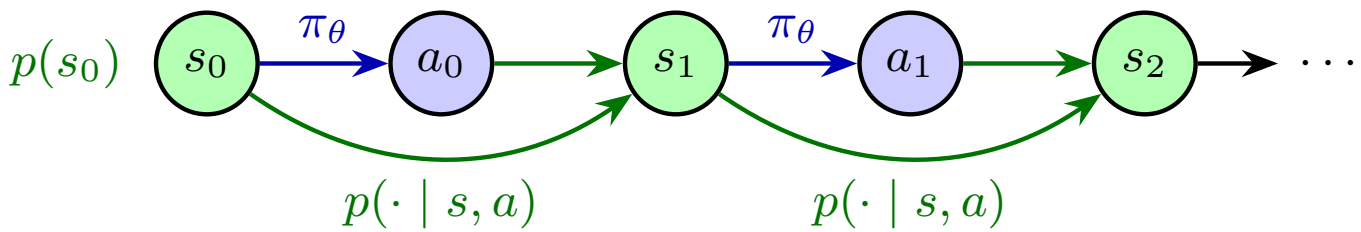


Central idea Iteratively improve π_θ by:

1. collecting trajectories (state–action–reward sequences) generated by the current policy π_θ,
2. estimating a gradient direction $\mathbf{g}$ that increases performance,
3. updating the policy parameters according to $\theta \leftarrow \theta + \alpha \mathbf{g}$, where $\alpha > 0$ is the learning rate.

Policy Gradient Methods: Trajectory Distribution

Trajectory $\tau = (s_0, a_0, s_1, a_1, \ldots)$: generated by alternating the policy and the environment.



Decomposing $p_\theta(\tau)$ Chain rule of probability, then the Markov assumptions: the action depends only on s_t (policy), the next state only on (s_t, a_t) (MDP dynamics):

$$p_\theta(\tau) = p(s_0)\, p(a_0 \mid s_0)\, p(s_1 \mid s_0, a_0)\, p(a_1 \mid s_0, a_0, s_1) \cdots \quad \text{(chain rule)}$$

$$= p(s_0) \prod_{t=0}^{\infty} \pi_\theta(a_t \mid s_t)\; p(s_{t+1} \mid s_t, a_t) \quad \text{(Markov)}$$

Only the blue factors depend on θ; the dynamics are θ-independent.

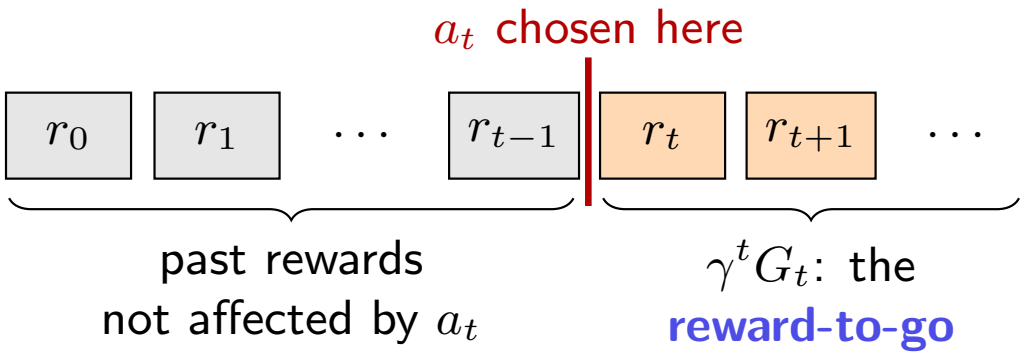
Policy Gradient Methods: Objective

Objective: maximize the expected discounted reward of a whole trajectory:

$$J(\theta) = \mathbb{E}_{\tau \sim p_\theta}[R(\tau)], \qquad R(\tau) = \sum_{t=0}^{\infty} \gamma^t r(s_t, a_t) = G_0,$$

i.e. $R(\tau)$ is the return G_t counted from the start, $t = 0$.

Splitting $R(\tau)$ at time t: $R(\tau) = \sum_{t'<t} \gamma^{t'} r_{t'} \; + \; \gamma^t \, G_t$



Causality: a_t can only influence rewards from time t on.
$\rightsquigarrow$ In the gradient, a_t will only be credited with G_t.

Consider

$$F(\theta) = \mathbb{E}_{\xi \sim p_\theta(\xi)}[f(\xi)].$$

The gradient of this parameterization is

$$\begin{aligned}
\nabla_\theta F(\theta) &= \int f(\xi)\, \nabla_\theta p_\theta(\xi)\, d\xi \\
&= \int p_\theta(\xi)\, f(\xi)\, \nabla_\theta \log p_\theta(\xi)\, d\xi \\
&= \mathbb{E}_{\xi \sim p_\theta(\xi)}[\, f(\xi)\, \nabla_\theta \log p_\theta(\xi)\,].
\end{aligned}$$

We conclude that

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta}\big[\, \nabla_\theta \log \pi_\theta(a \mid s)\, R(\tau)\,\big]. \tag{3}$$

Policy Gradient Methods: REINFORCE algorithm [Wil92]

Require: initial policy parameter $\theta \in \mathbb{R}^d$, step size $\alpha > 0$, baseline $b(\cdot)$

for each episode **do**
  Generate an episode $(s_0, a_0, r_0), \ldots, (s_T, a_T, r_T)$ following π_θ
  for $t = 0$ to T **do**
    $G_t \leftarrow \sum_{i=t}^{T} \gamma^{i-t} r_i$
    $\hat{A}_t \leftarrow G_t - b(s_t)$
    $\theta \leftarrow \theta + \alpha \, \hat{A}_t \nabla_\theta \log \pi_\theta(a_t \mid s_t)$
  end for
end for

Baseline $b(s_t)$: subtracted to reduce variance; it leaves the gradient unbiased.

Advantage: $A_\pi(s, a) = q_\pi(s, a) - v_\pi(s)$, how much better action a is than the policy's average action in s. With the common choice $b(s_t) \approx v_\pi(s_t)$, $\hat{A}_t$ is a one-sample **estimate** of $A_\pi(s_t, a_t)$ (since $\mathbb{E}[G_t \mid s_t, a_t] = q_\pi(s_t, a_t)$): $\hat{A}_t > 0$ means a_t did better than average.

Bernoulli policy (binary action):

$$\pi_\theta(a \mid s) = p_\theta(s)^a \, [1-p_\theta(s)]^{1-a}, \quad \nabla_\theta \log \pi_\theta(a \mid s) = (a - p_\theta(s)) \, \nabla_\theta z_\theta(s),$$

with $p_\theta(s) = \sigma(z_\theta(s))$. If $z_\theta(s) = \theta^\top \phi(s)$,

$$\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta}\big[(a - p_\theta(s)) \, \phi(s) \, R(s,a)\big].$$

Two-layer policy network:

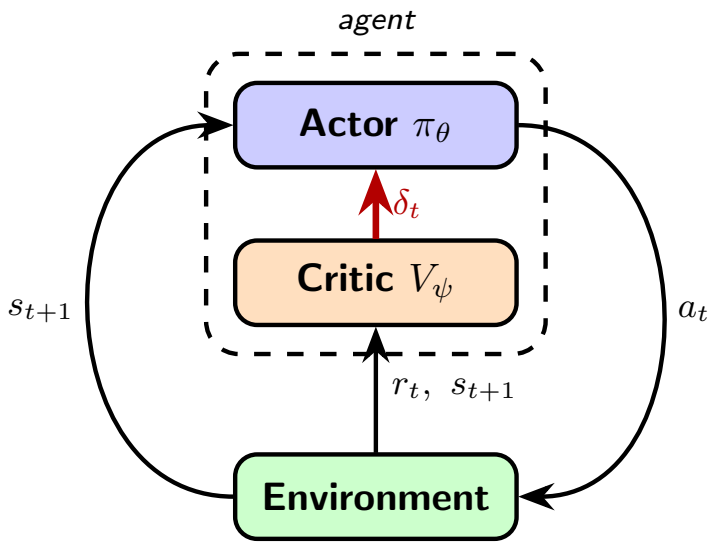
$$h_t = \sigma(W_1 x_t), \quad z_t = W_2 h_t, \quad p_t = \mathrm{softmax}(z_t),$$

$$\nabla_{z_t} \log \pi_\theta(a_t \mid s_t) = y_t - p_t, \quad \nabla_{W_2} \log \pi_\theta(a_t \mid s_t) = (y_t - p_t) \, h_t^\top.$$

Problem with REINFORCE: the update $\theta \leftarrow \theta + \alpha\,(G_t - b(s_t))\,\nabla_\theta \log \pi_\theta(a_t \mid s_t)$ needs the full return G_t: we must wait until the episode ends, and G_t is very noisy.

Idea: learn a second network that **predicts** returns, and use it to judge each action after a single step.

- **Actor** $\pi_\theta(a \mid s)$: the policy. It **chooses** the action a_t (policy-based).
- **Critic** $V_\psi(s) \approx v_{\pi_\theta}(s)$: predicts the expected return of the **current** actor from state s. It **evaluates** what happened (value-based).



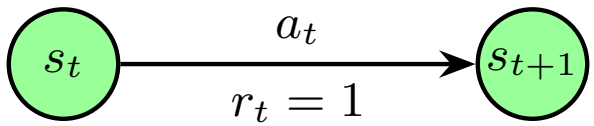
The critic's feedback to the actor is a single number per step, the **TD error** δ_t.

TD Error: the Critic's Feedback

After one transition (s_t, a_t, r_t, s_{t+1}), compare the prediction made **before** acting with a better one made **after**:

$$\delta_t \;=\; \underbrace{r_t + \gamma\, V_\psi(s_{t+1})}_{\substack{\text{TD target: reward received} \\ + \text{ estimate of the rest}}} \;-\; \underbrace{V_\psi(s_t)}_{\substack{\text{prediction} \\ \text{before acting}}}$$

$V_\psi(s_t) = 5$ $\qquad$ $V_\psi(s_{t+1}) = 6$



target: $1 + 0.9 \cdot 6 = 6.4$

$\delta_t = 6.4 - 5 = +1.4$

($\gamma = 0.9$): better than expected

Why it works: by the Bellman equation, a perfect critic ($V_\psi = v_\pi$) has $\mathbb{E}[\delta_t \mid s_t] = 0$.

So $\delta_t \neq 0$ is a surprise: the step went better ($\delta_t > 0$) or worse ($\delta_t < 0$) than predicted.

Actor-Critic Updates

At each step, the critic minimizes the squared TD error, while the actor performs a gradient ascent using the advantage estimate:

$$\psi \leftarrow \psi - \beta \, \nabla_\psi \left(\tfrac{1}{2} \, \delta_t^2 \right)$$
$$\theta \leftarrow \theta + \alpha \, \hat{A}_t \, \nabla_\theta \log \pi_\theta(a_t \mid s_t) + \lambda_H \nabla_\theta \mathcal{H}(\pi_\theta(\cdot \mid s_t)).$$

α, β are learning rates, $\mathcal{H}$ is the entropy of the policy, and λ_H controls its weight (encouraging exploration).

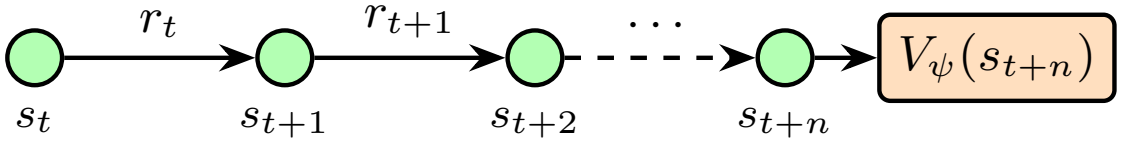
Multi-step Advantages and GAE

Setting: run the policy for T steps and store the rewards $r_0, \ldots, r_{T-1}$.

n-step return: from step t, use the next n stored rewards, then the critic:

$$G_t^{(n)} = \underbrace{r_t + \gamma r_{t+1} + \cdots + \gamma^{n-1} r_{t+n-1}}_{n \text{ stored rewards}} + \underbrace{\gamma^n V_\psi(s_{t+n})}_{\text{critic}}.$$

Advantage estimate: $\hat{A}_t^{(n)} = G_t^{(n)} - V_\psi(s_t)$.



$n = 1$: the TD error δ_t (low variance, biased).
$n = T - t$: all remaining rewards (unbiased, high variance).

GAE: average over all n, with weights $\propto \lambda^{n-1}$. This simplifies to

$$\hat{A}_t^{\mathsf{GAE}(\lambda)} = \sum_{k=0}^{T-t-1} (\gamma\lambda)^k \, \delta_{t+k}, \qquad \lambda \in [0, 1].$$

Relation to REINFORCE

REINFORCE uses the Monte Carlo return G_t as the learning signal. Actor-Critic replaces G_t by lower-variance (but possibly biased) advantage estimates.

This modification yields:

- Faster and more stable convergence
- Lower gradient variance
- Slight bias if V_ψ is imperfect

Implementation of the Critic

In practice, the critic $V_\psi(s)$ is implemented as a neural network mapping a state (or state-action pair) to a scalar prediction of expected return.

Typical structure:

$$V_\psi : \mathcal{S} \to \mathbb{R}, \qquad V_\psi(s_t) \approx \mathbb{E}_{\pi_\theta}[G_t \mid s_t].$$

Actor and critic often share an encoder ϕ:

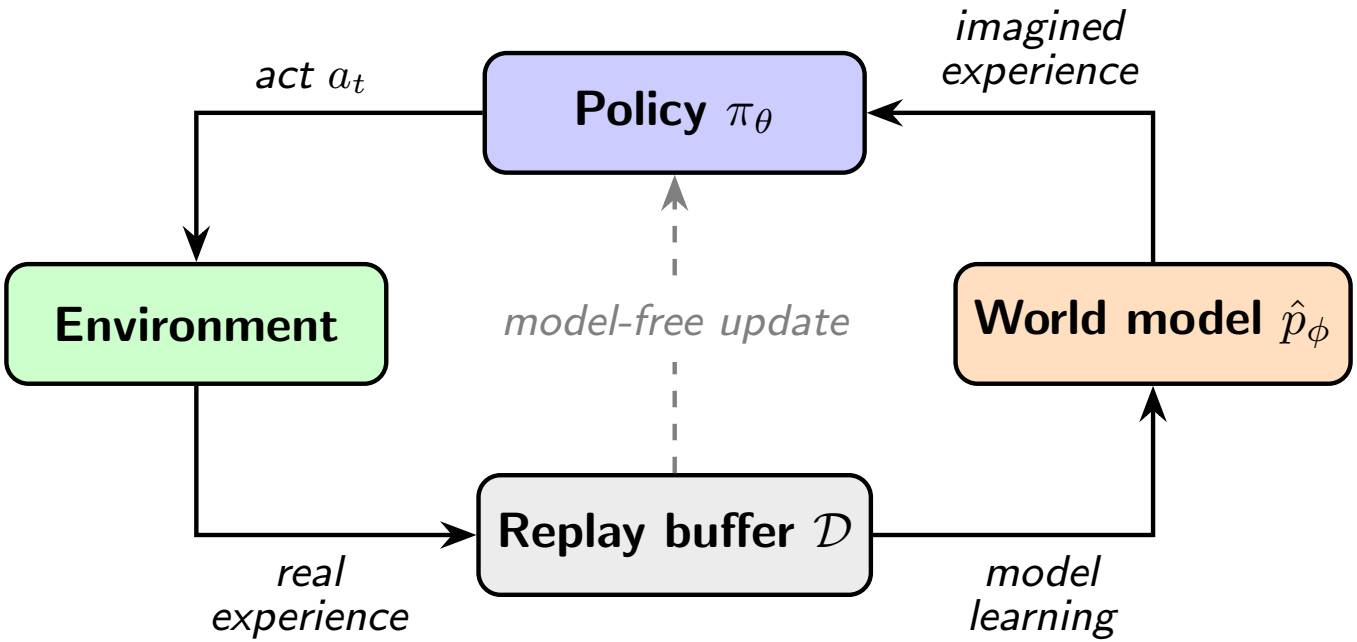
$$h_t = f_\phi(s_t), \quad \pi_\theta(a \mid s_t) = \mathsf{softmax}(W_\pi h_t), \quad V_\psi(s_t) = W_V h_t.$$

This improves sample efficiency, while using separate output heads ensures stable optimization.

Model-Free vs Model-Based RL

So far: model-free. DQN, REINFORCE and actor-critic learn Q, π or V directly from interaction; the dynamics $p(r, s' \mid s, a)$ are never estimated. $\rightsquigarrow$ very sample-hungry (standard Atari budget: 200M frames $\approx$ 38 days of play per game).

Model-based: learn a **world model** $\hat{p}_\phi(r, s' \mid s, a)$ from data, then improve the policy on **imagined** experience.



DreamerV2 [HLNB20]: Overview

Idea: learn a compact **latent** world model from pixels and train an actor-critic **purely inside the model** ("dreaming").

Three processes run concurrently:

1. **Data collection:** act in the environment with the current actor, store episodes in a replay buffer $\mathcal{D}$.
2. **World model learning:** fit latent dynamics ϕ on sequences sampled from $\mathcal{D}$.
3. **Behavior learning:** train actor π_θ and critic V_ψ on imagined latent trajectories.

Remark 2 (Two design choices worth noting)

- *The policy never sees pixels: it acts on a latent state s_t*
- *Actor/critic gradients do not flow into the world model.*

World Model: Recurrent State-Space Model (RSSM)

Data: images x_t, actions a_t, rewards r_t, discount flags γ_t (0 at episode end).

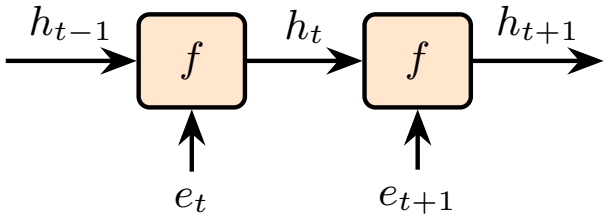
Latent **model state** $s_t := (h_t, z_t)$: deterministic memory h_t (GRU) + stochastic z_t.

Definition 3 (RSSM components (all parameterized by ϕ))

Recurrent model:	$h_t = f_\phi(h_{t-1}, z_{t-1}, a_{t-1})$
Representation model:	$z_t \sim q_\phi(z_t \mid h_t, x_t)$
Transition predictor:	$\hat{z}_t \sim p_\phi(\hat{z}_t \mid h_t)$
Image predictor:	$\hat{x}_t \sim p_\phi(\hat{x}_t \mid h_t, z_t)$
Reward predictor:	$\hat{r}_t \sim p_\phi(\hat{r}_t \mid h_t, z_t)$
Discount predictor:	$\hat{\gamma}_t \sim p_\phi(\hat{\gamma}_t \mid h_t, z_t)$

In brief: Gated Recurrent Unit (GRU)

Recurrent network: a memory h_t updated from inputs e_t, $\quad h_t = f(h_{t-1}, e_t)$.
A plain RNN, $h_t = \tanh(W[h_{t-1}, e_t])$, overwrites the memory at every step $\Rightarrow$ it forgets quickly and gradients vanish.

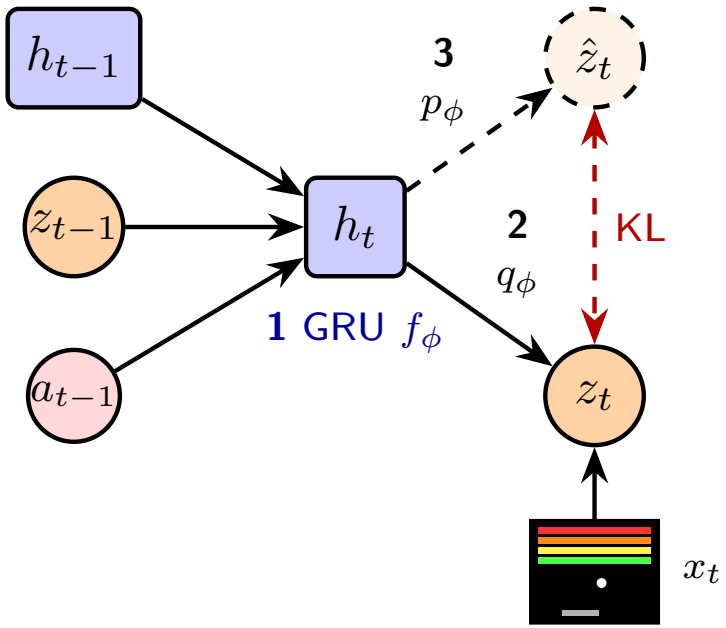


GRU: gates in $(0, 1)$ decide when to keep and when to overwrite the memory.

$$u_t = \sigma\big(W_u[h_{t-1}, e_t]\big) \qquad \text{update gate}$$
$$g_t = \sigma\big(W_g[h_{t-1}, e_t]\big) \qquad \text{reset gate}$$
$$\tilde{h}_t = \tanh\big(W[g_t \odot h_{t-1},\, e_t]\big) \qquad \text{candidate memory}$$
$$h_t = (1 - u_t) \odot h_{t-1} + u_t \odot \tilde{h}_t \qquad \text{mix old and new}$$

- $u_t \approx 0$: copy the memory. $u_t \approx 1$: overwrite it with candidate $\tilde{h}_t$.
- In DreamerV2: $h_t = \mathrm{GRU}(h_{t-1}, e_t)$ with input $e_t = (z_{t-1}, a_{t-1})$.

RSSM (1/2): Building the Latent State



1. Recurrent model (GRU)
h_t: deterministic summary of $x_{<t}, a_{<t}$. "What I remember."

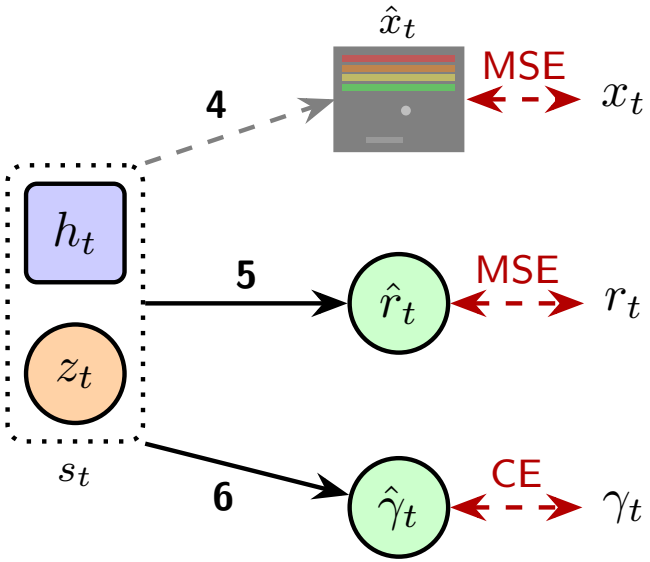
2. Representation model (posterior)
CNN-encodes x_t, combines it with h_t. z_t: what the frame adds. "What I see." Needs real frames: used on replayed data and while acting.

3. Transition predictor (prior)
Guesses z_t from h_t alone, before seeing x_t. "What I expect to see." The only option in imagination (no frames there).

Remark 3 (Predict, then correct (as in a Kalman filter))

Prior predicts, posterior corrects with x_t, KL teaches the prior to anticipate the posterior. h_t carries information reliably over many steps; z_t captures uncertainty (several possible futures).

RSSM (2/2): Prediction Heads



4. Image predictor (transposed CNN)
Reconstructs the frame (pixel MSE). Forces s_t to describe the whole scene: the main training signal.

5. Reward predictor (MLP)
Regresses r_t (MSE). Supplies the rewards of imagined trajectories.

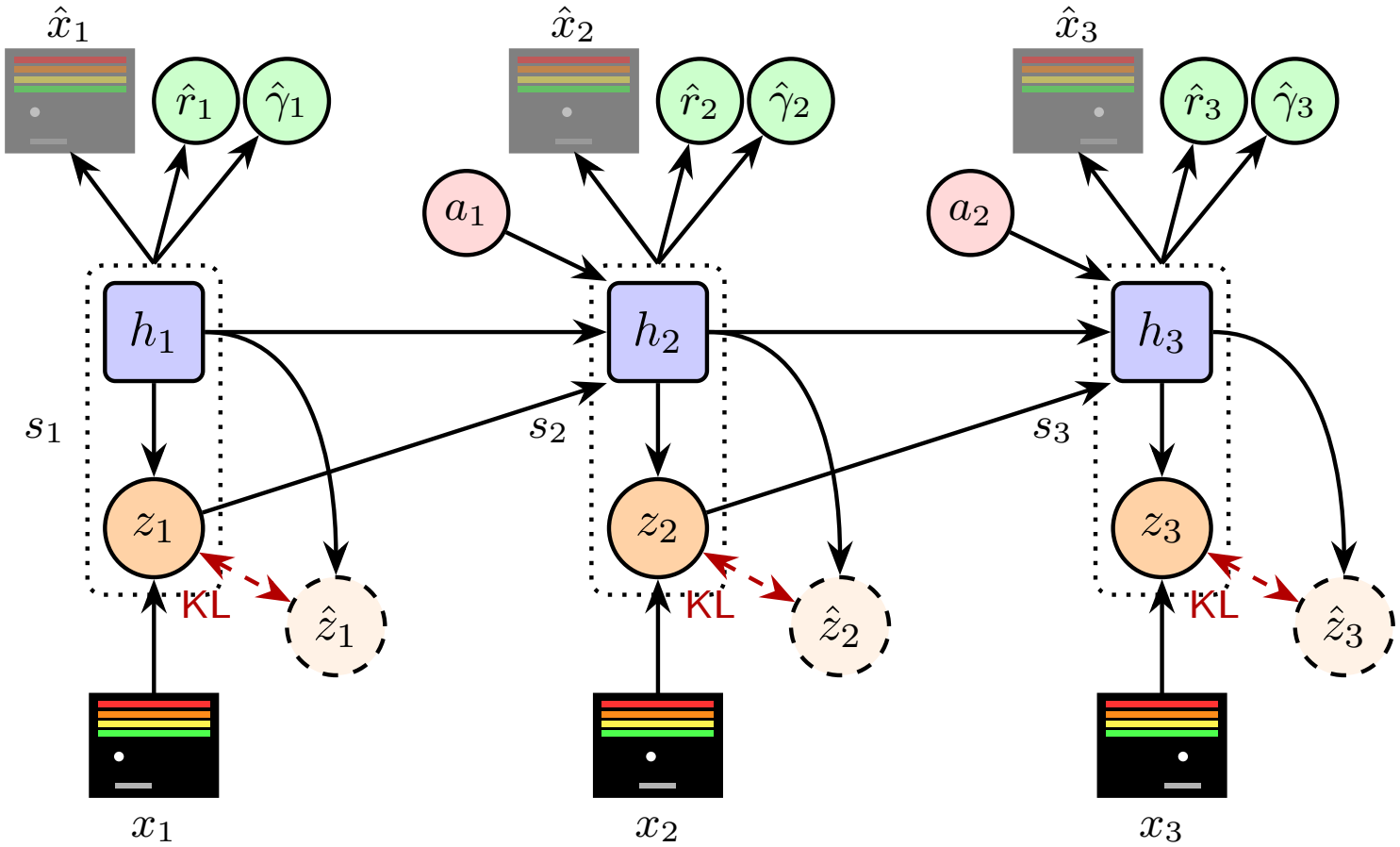
6. Discount predictor (MLP)
Does the episode continue? Binary cross-entropy, with $\gamma_t = 0$ at the end. Imagined returns stop at terminal states.

Remark 4 (Together: a learned MDP in latent space)

$$\Big(\underbrace{\{s = (h, z)\}}_{S},\ A,\ \underbrace{f_\phi + p_\phi(\hat{z} \mid h)}_{P},\ \underbrace{p_\phi(\hat{r} \mid s)}_{R},\ \underbrace{p_\phi(\hat{\gamma} \mid s)}_{\gamma} \Big)$$

mirrors the MDP 5-tuple (S, A, P, R, γ).

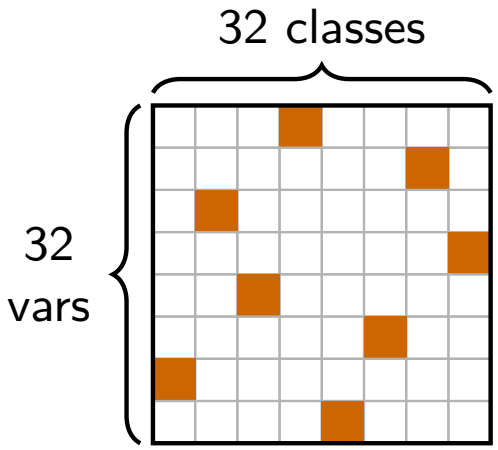
World Model: Unrolled over Time



■ deterministic • stochastic posterior (sees x_t) dashed: prior $\hat{z}_t$ (no x_t)

All predictions use the full model state $s_t = (h_t, z_t)$.

Discrete Latent Representations



one-hot per row $\rightarrow$ sparse vector,
32 ones out of 1024

DreamerV2 latents: z_t is a vector of **32 categorical variables** with 32 classes each (DreamerV1 used Gaussians).

Straight-through gradients. Sampling is not differentiable. With class probabilities p:

$$z = \underbrace{\text{onehot}(\text{sample})}_{\text{forward value}} + p - \text{sg}(p),$$

forward: the sample; backward: ∇p (sg = stop-gradient).

Remark 5 (Why discrete? (hypotheses from the paper))

Categorical priors fit the aggregate posterior well, codes are sparse, and they suit non-smooth dynamics (objects appearing suddenly). Large empirical gain on Atari.

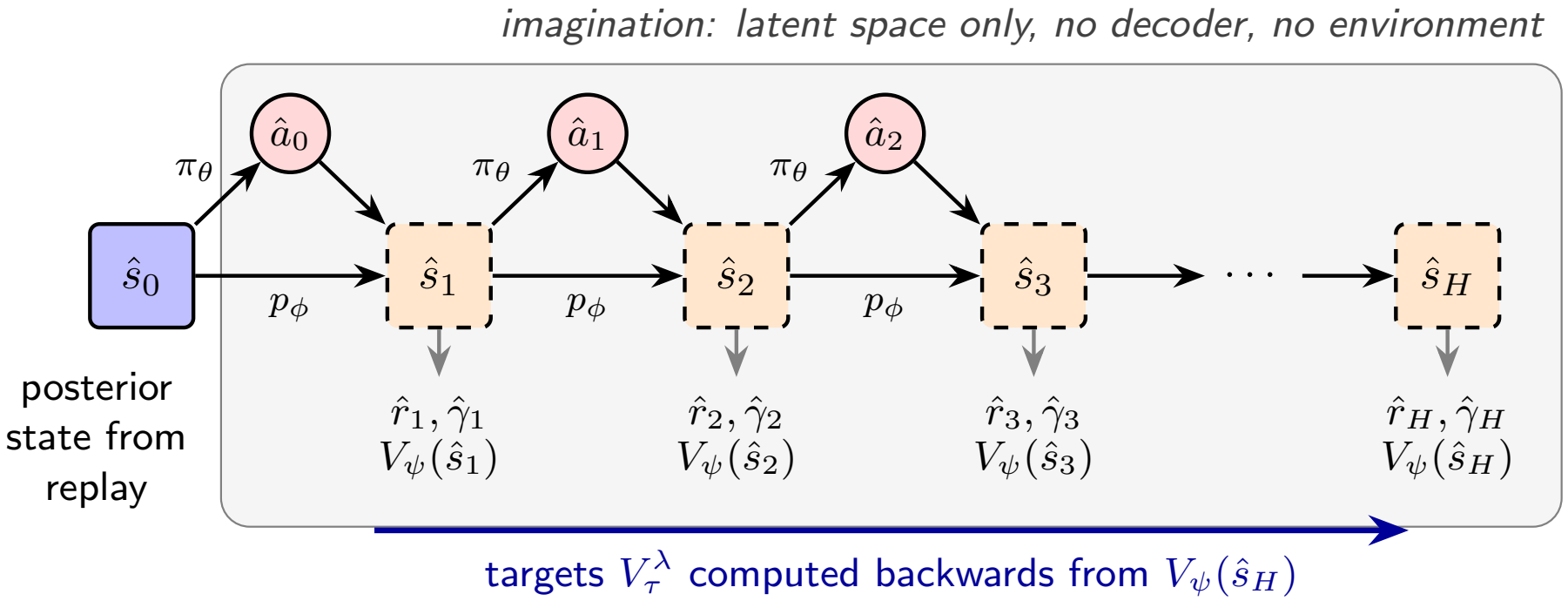
For sequences $(x_t, a_t, r_t, \gamma_t)_{t=1}^{T} \sim \mathcal{D}$ and posterior states s_t, minimize

$$\mathcal{L}(\phi) = \mathbb{E}_{q_\phi}\Big[\sum_{t=1}^{T}\Big(\underbrace{-\ln p_\phi(x_t \mid s_t)}_{\text{image}}\underbrace{-\ln p_\phi(r_t \mid s_t)}_{\text{reward}}\underbrace{-\ln p_\phi(\gamma_t \mid s_t)}_{\text{discount}} + \beta\ \mathrm{KL}\big[\underbrace{q_\phi(z_t \mid h_t, x_t)}_{\text{representation}} \,\big\|\, \underbrace{p_\phi(z_t \mid h_t)}_{\text{transition}}\big]\Big)\Big].$$

$\rightsquigarrow$ a sequential VAE (ELBO): reconstruct observations, keep latents predictable from the past.

Every RSSM component is trained by this loss: heads via the log-likelihoods; representation and transition via the KL (and q_ϕ also via $\mathbb{E}_{q_\phi}$, since s_t is sampled from it); the recurrent model through h_t, which enters every term (backprop through time).

Behavior Learning in Imagination



- Start from **every** posterior state of the training batch, roll out $H = 15$ steps with the prior and the actor, all in parallel.
- Same actor-critic as before: actor $\pi_\theta(\hat{a}_\tau \mid \hat{s}_\tau)$, critic $V_\psi(\hat{s}_\tau)$ are trained; the world model ϕ is frozen here.

DreamerV2: Algorithm

Require: replay $\mathcal{D}$ (random episodes), world model ϕ, actor θ, critic ψ, horizon H

while not converged **do**
  for update step $c = 1, \ldots, C$ **do**
    Sample batch of sequences $(x_t, a_t, r_t, \gamma_t)$ from $\mathcal{D}$
    Compute posterior states $s_t = (h_t, z_t)$
    Update ϕ with $\mathcal{L}(\phi)$ ▷ world model
    From each s_t, imagine $(\hat{s}_\tau, \hat{a}_\tau)_{\tau=1}^{H}$ with p_ϕ and π_θ ▷ dreaming
    Predict $\hat{r}_\tau, \hat{\gamma}_\tau$ and compute V_τ^λ using the target critic
    Update θ with $\mathcal{L}(\theta)$ and ψ with $\mathcal{L}(\psi)$ ▷ behavior
  end for
  for environment step $t = 1, \ldots, T$ **do**
    Update s_t from (s_{t-1}, a_{t-1}, x_t) with f_ϕ and q_ϕ
    Sample $a_t \sim \pi_\theta(a \mid s_t)$, execute it, observe r_t, x_{t+1}
  end for
  Add the new experience to $\mathcal{D}$
end while

Atari (55 games, 200M frames, single GPU):

- First agent to reach human-level performance on the Atari benchmark by learning behaviors inside a separately trained world model.
- With the same compute and wall-clock budget, outperforms the top single-GPU model-free agents such as DQN.

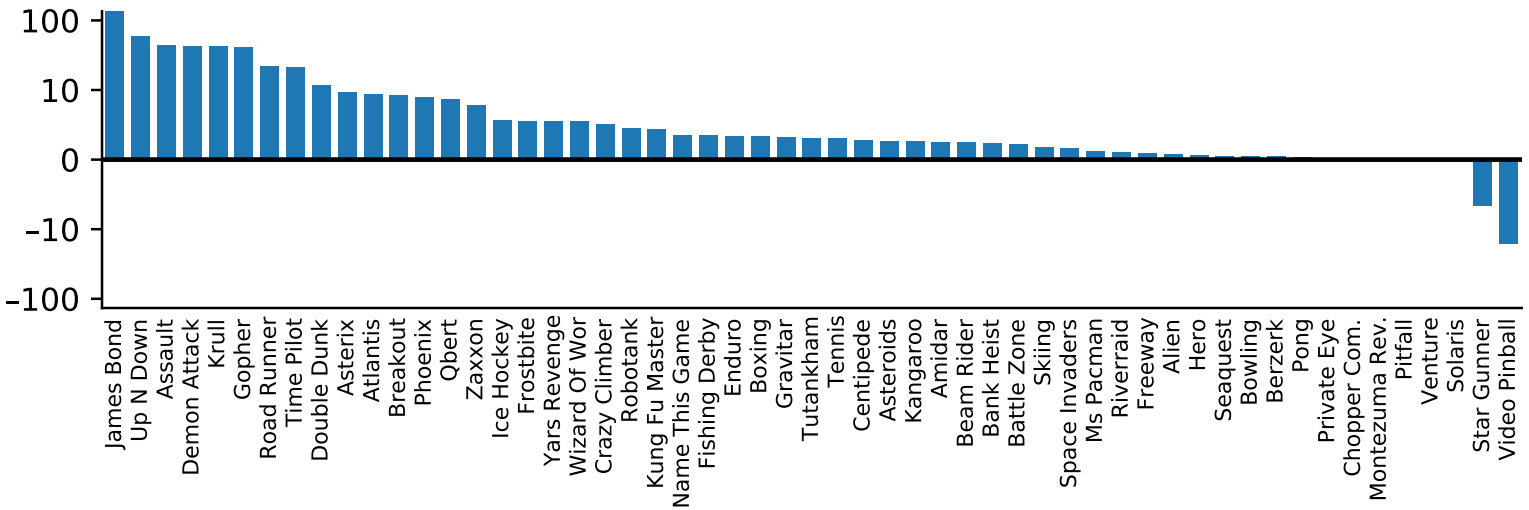


Figure: Atari agent comparison. The bars show the difference vs DQN in gamer normalized scores at 200M steps.



Danijar Hafner, Timothy Lillicrap, Mohammad Norouzi, and Jimmy Ba.

Mastering atari with discrete world models.

arXiv preprint arXiv:2010.02193, 2020.



Ronald J Williams.

Simple statistical gradient-following algorithms for connectionist reinforcement learning.

Machine learning, 8(3):229–256, 1992.